The Host

The object that holds the whole application and manages its life

1 What is it?

The **Host** is the object that **holds the whole application and manages its life**: it starts it, keeps it running, and stops it properly.

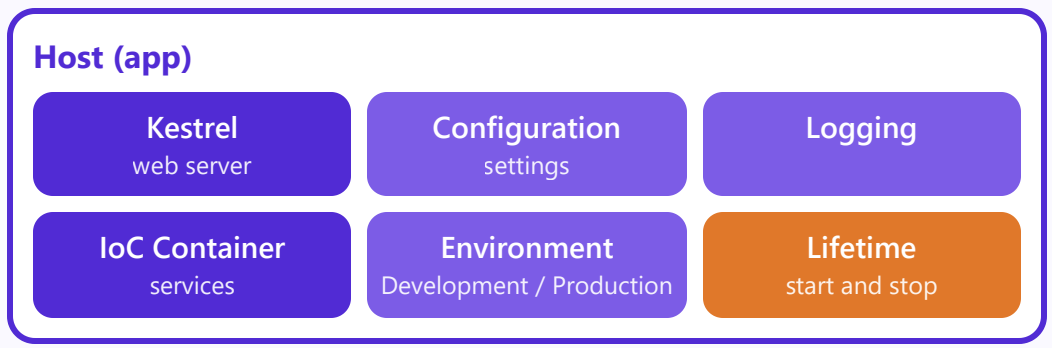
2 Why does it exist?

An application is more than code that answers requests. It needs a **web server** to accept connections, **settings**, **logging**, a **services container** (IoC Container), and a way to stop without cutting running requests. All of this is gathered and managed in one place: the Host.

3 Who creates it, and when?

We do not create it by hand. It is created when we call `Build()`, and `app` **is** the Host.

THE HOST HOLDS EVERYTHING



IN PROGRAM.CS

```
// prepare the Host
var builder = WebApplication.CreateBuilder(args);

// the Host is created
var app = builder.Build();

// it starts and keeps running
app.Run();
```

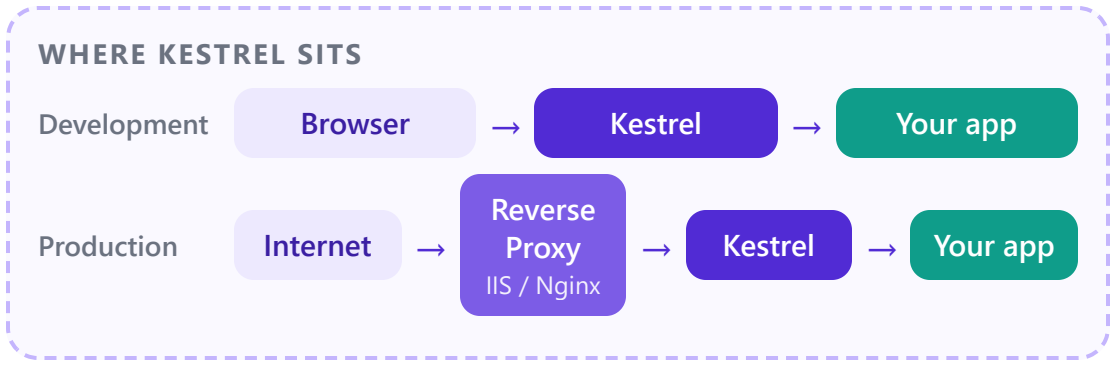
- `app` is of type **WebApplication**, and it is the Host itself.
- This style is called the **Minimal Hosting Model** (since .NET 6).
- Before it, there were two files: `Program.cs` and `Startup.cs`, and the Host was created with `Host.CreateDefaultBuilder` then `ConfigureWebHostDefaults`.
- The idea is the same. Only the writing became shorter.

What the Host holds

Prepared for us by `CreateBuilder`, with no code from us

Thing	What is prepared
Web server	Kestrel, the default server
Configuration	`appsettings.json`, then `appsettings.{Environment}.json`, then User Secrets (in Development), then Environment Variables, then command-line args
Logging	Ready providers (Console, Debug, and others)
IoC Container	The container where we register services through `builder.Services`
Environment	`Development` or `Production`, from `ASPNETCORE_ENVIRONMENT`

> This is why `appsettings.json` and `launchSettings.json` take effect without any code of ours reading them.



Kestrel: the web server

1 What is it?

A program that opens a **port** on the network (like 5149), receives HTTP requests, and passes them to our application.

2 Why does it exist?

The app does not talk to the network directly. Kestrel reads the raw request, turns it into an **HttpContext**, and sends it into the pipeline.

3 Where is it set?

In development the port comes from `applicationUrl` in `launchSettings.json`. Kestrel is built into the app, so `dotnet run` works without installing any server.

4 In production

Usually another server sits in front of it, called a **Reverse Proxy** (like IIS or Nginx). It receives requests from the internet and forwards them to Kestrel.

- **In-process:** the app runs inside the IIS process (the default on IIS).
- **Out-of-process:** IIS forwards requests to Kestrel in a separate process.

The Host lifetime

Start, run, and stop properly

Started → **Running** → Stopping → Stopped

1 What is it?

The three phases the application goes through: it **starts**, it **runs**, and it **stops**.

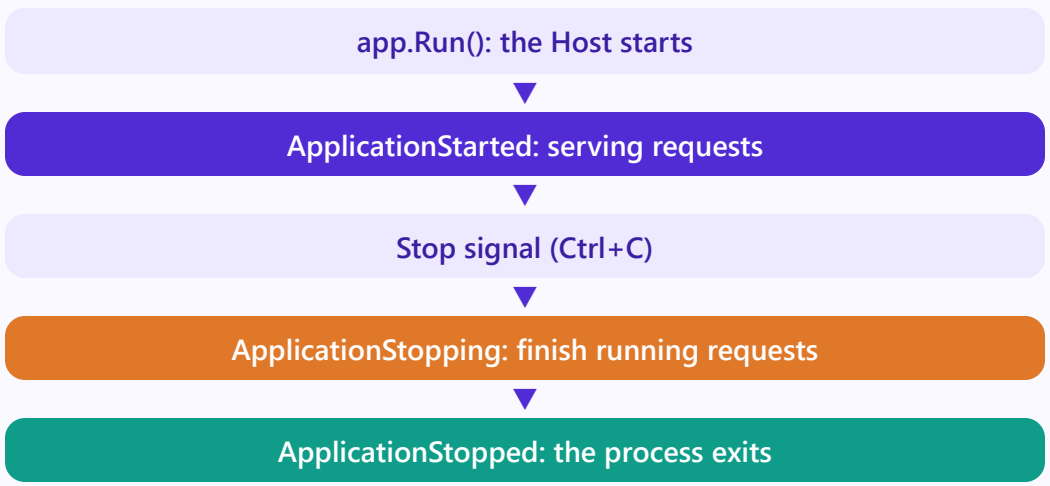
2 Why does it matter?

When you stop the app (for example with Ctrl+C), it is not cut suddenly. It stops accepting new requests and finishes the running ones. This is a **graceful shutdown**. We can also run our own code in each phase, such as cleaning up resources.

3 What does app.Run() do?

It starts the Host and **blocks the thread** until a stop command arrives. That is why no code after it runs.

TIMELINE



THE LIFETIME PROPERTY

```
var app = builder.Build();

app.Lifetime.ApplicationStarted.Register(() =>
    Console.WriteLine("Host: started"));

app.Lifetime.ApplicationStopping.Register(() =>
    Console.WriteLine("Host: stopping"));

app.Lifetime.ApplicationStopped.Register(() =>
    Console.WriteLine("Host: stopped"));
```

> **Try it:** run the app and you see `Host: started`. Press Ctrl+C and you see `Host: stopping` then `Host: stopped`.

HttpContext

Everything about one request and its response

1 What is it?

An object that represents **one request with all its details, and its response**. It carries the incoming data (Request), the answer we will send (Response), the user, and more.

2 Why does it exist?

Instead of passing dozens of parameters from place to place, **one object** that contains everything is passed. It is also how the parts of the app (the middlewares) share information about the same request.

3 Who creates it, and when?

- **Kestrel** creates it when each request arrives, so every request gets a **new** HttpContext.
- It travels through the whole pipeline, then it is **disposed** when the request ends.
- It never lives longer than the request.

HOW DO WE REACH IT?

Where	How
Minimal API	A parameter: `(HttpContext context) => ...`
Middleware	The `context` parameter in `app.Use(async (context, next) => ...)`
Controller	A ready property named `HttpContext`
Regular class	The `IHttpContextAccessor` interface, after registering it with `AddHttpContextAccessor()`

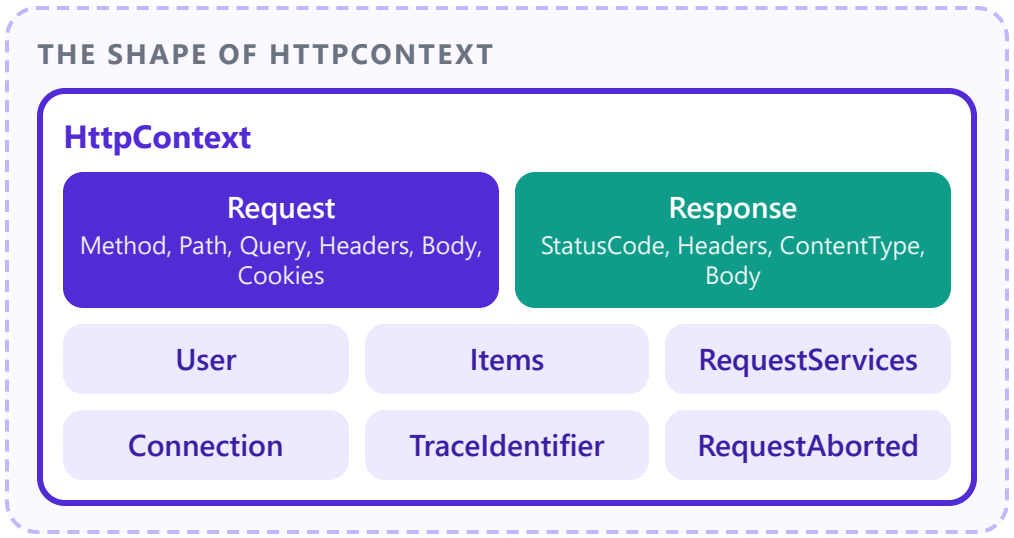
THE LIFE OF ONE HTTPCONTEXT



What is inside HttpContext

The main properties

Property	What it is	Examples
`Request`	The incoming request	`Method`, `Path`, `Query`, `Headers`, `Body`, `Cookies`
`Response`	The answer we send	`StatusCode`, `Headers`, `ContentType`, `Body`
`User`	The current user (`ClaimsPrincipal`)	Used in authentication
`Items`	A dictionary to share data between middlewares in the same request	`context.Items["key"]`
`RequestServices`	Gets services from the IoC Container	Dependency Injection
`Connection`	Connection information	`RemoteIpAddress`
`TraceIdentifier`	A unique id for the request	Useful in logging
`RequestAborted`	A `CancellationToken` that is cancelled if the client disconnects	



Experiment 1: read the Request

Everything the request carries is on `context.Request`

CODE

```
app.MapGet("/info", (HttpContext context) => new
{
    context.Request.Method,
    Path = context.Request.Path.Value,
    Query = context.Request.QueryString.Value,
    UserAgent = context.Request.Headers.UserAgent.ToString(),
    Ip = context.Connection.RemoteIpAddress?.ToString(),
    context.TraceIdentifier
});
```

NOTES

- `Method` is the HTTP Verb (GET).
- `Path` and `Query` come from the URL.
- Send the request again: `TraceIdentifier` changes, because **every request has a new HttpContext**.
- Change the URL and watch the values change.

OPEN /INFO?X=1 AND YOU GET JSON

```
{"method":"GET","path":"/info","query":"?x=1",
 "userAgent":"...","ip":"::1",
 "traceIdentifier":"0HN...:00000001"}
```

WHERE EACH PART OF THE URL GOES

GET	http://localhost:5149	/info	?x=1
Request.Method	Request.Scheme + Request.Host	Request.Path	Request.QueryString

Experiment 2: write the Response

The answer is built on `context.Response`

CODE

```
app.MapGet("/write", async (HttpContext context) =>
{
    context.Response.StatusCode = 201;
    context.Response.Headers["X-Course"] = "ASP.NET Core";
    context.Response.ContentType = "text/plain; charset=utf-8";
    await context.Response.WriteAsync("Written directly");
});
```

Order matters: set the headers and the status code **before** writing the body. Once writing starts, the headers are sent and can no longer change.

THE ORDER OF WRITING

1. StatusCode
201
→
2. Headers
X-Course
→
3. Body
WriteAsync(...)

Headers are sent when the body starts. After step 3 you cannot change 1 or 2.

NOTES

- Open `/write` and look at the Network tab: the status is `201 Created` and the header `X-Course` is there.
- Normally we do not write the response by hand. When we `return "Hello"` or an object, ASP.NET Core writes the response for us. This is what happens underneath.

Important notes and summary

What to avoid, and what to remember

IMPORTANT NOTES

> **One request only.** Do not store HttpContext in a `static` variable or use it after the request ends.

> **Not thread-safe.** Do not use it from several threads at the same time.

> **In a regular class** use `IHttpContextAccessor` . Do not try to pass the HttpContext by hand everywhere.

TRY IT YOURSELF

- Add a route `/headers` that returns all the headers of the request.
- Add a route that returns the `TraceIdentifier` , call it several times, and notice what changes.
- Print a message in `ApplicationStopping` , press Ctrl+C, and check that it appears.

SUMMARY

Concept	What it is	Who creates it
`Host (WebApplication)`	Holds the app and manages its life	`builder.Build()`
`Kestrel`	The web server that receives requests	The Host
`HttpContext`	Everything about one request and its response	Kestrel, for each request